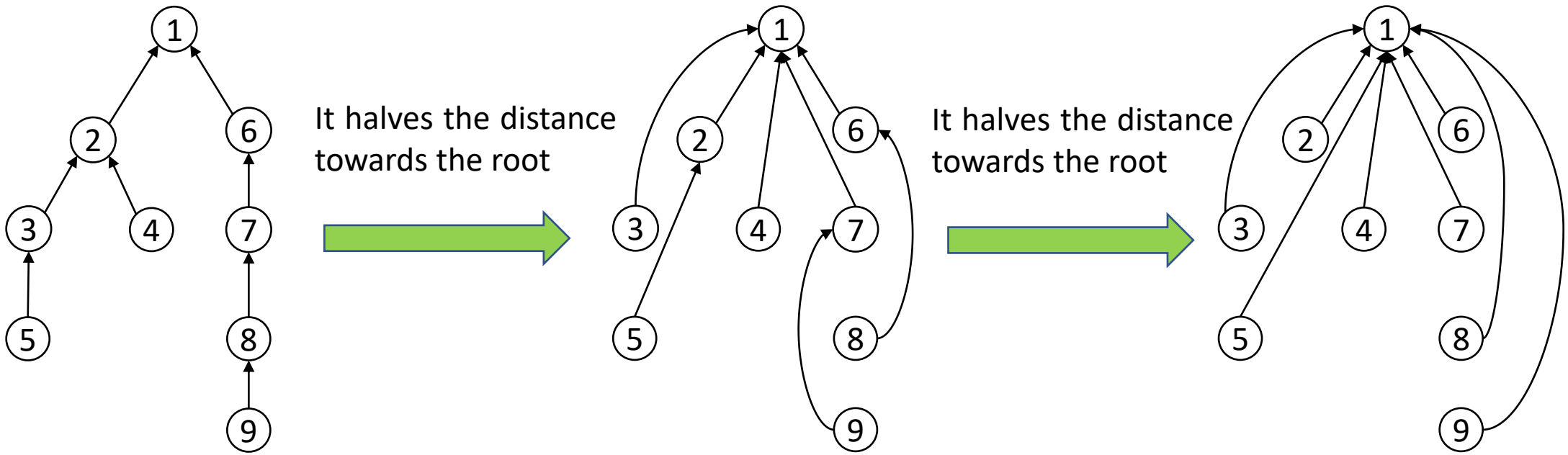
Thinking in parallel

How to design algorithms for multi-processor systems

Basic techniques

Basic techniques: Pointer jumping

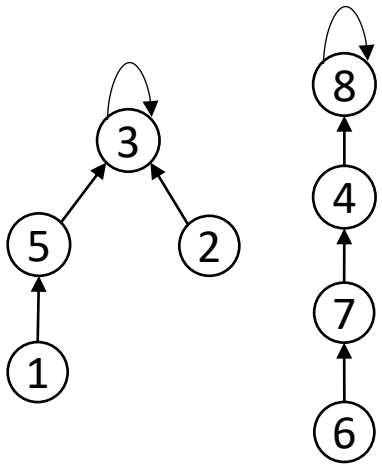
- The technique of *pointer jumping*, a.k.a. *path doubling*, allows for the fast processing of data stored in the form of a set of rooted trees, where each node of the tree has a direct pointer to its parent
- It updates the parent of a node of the tree with its grandparent recursively



Finding the roots of a forest

FINDING THE ROOTS OF A FOREST
Input: An array P of n elements, where element i corresponds to node i in a forest and $P[i]$ contains the parent of i (we assume a root points to itself)
Output: An array R of n elements such that $R[i]$ contains the root of node i

EXAMPLE:



index	1	2	3	4	5	6	7	8
P	5	3	3	8	3	7	4	8

index	1	2	3	4	5	6	7	8
R	3	3	3	8	3	8	8	8

An asymptotic time-optimal sequential algorithm

FINDING THE ROOTS OF A FOREST
Input: An array P of n elements, where element i corresponds to node i in a forest and $P[i]$ contains the parent of i (we assume a root points to itself)
Output: An array R of n elements such that $R[i]$ contains the root of node i

AN $O(n)$-TIME SEQUENTIAL ALGORITHM

1. Identify the roots (all indices i such that $P[i] = i$)
2. Compute pointers to children for all nodes
3. for each root r of a tree
4. visit the tree rooted at r and set R[i] ← r when visiting node i

it is asymptotically optimal, …
… but it is not parallelizable

This is why we need the pointer jumping technique!

A parallel solution

FINDING THE ROOTS OF A FOREST
Input: An array P of n elements, where element i corresponds to node i in a forest and $P[i]$ contains the parent of i (we assume a root points to itself)
Output: An array R of n elements such that $R[i]$ contains the root of node i

ALGORITHM Pointer_Jumping

1. **for** $1 \leq$ i $\leq$ n **pardo**
2. R[i] $\leftarrow$ P[i]
3. **while**(R[i] $\neq$ R[R[i]]) **do**
4. R[i] $\leftarrow$ R[R[i]]

- h: maximum among heights of all trees in the forest
- Time $O(\log h)$
- Work $O(n \log h)$ (not work optimal)
- CREW PRAM algorithm
  - No concurrent write on same variable
  - Concurrent reads on same variable: if two nodes have the same parent j, then we simultaneously access $R[j]$ (steps 3-4)

Correctness proof

THEOREM: Given a forest of rooted trees with n nodes, the algorithm Pointer_Jumping computes, for each node i, the root of i. The algorithm runs on a CREW PRAM in time $O(\log h)$ and work $O(n \log h)$, where h is the maximum among the heights of all trees in the forest.

PROOF (only correctness): We show that after k iterations of the while loop, for every node i:

1. if the hop-distance from i to its root is at most 2^k , then $R[i]$ is the root of i,
2. if the hop-distance from i to its root is $> 2^k$, then $R[i]$ is the 2^k-th ancestor of i.

The proof is by induction on k.

BASE CASE $k = 0$: trivial.

INDUCTIVE CASE $k + 1$: Assume node i is at a hop-distance $> 2^k$ from its root, otherwise we use inductive hypothesis. By the induction hypothesis, node $R[i]$ is the 2^k-th ancestor of i. By induction

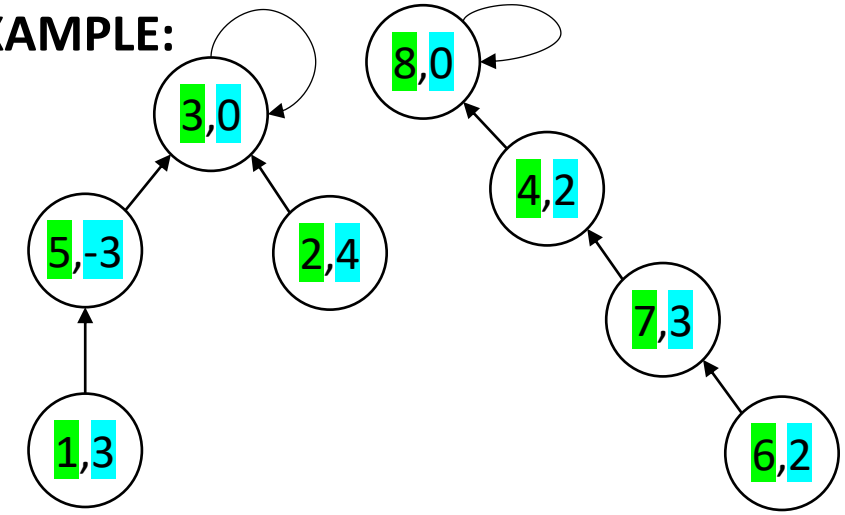
a. if the hop-distance from $R[i]$ to its root is at most 2^k, then $R[R[i]]$ is the root of $R[i]$,
b. if the hop-distance from $R[i]$ to its root is $> 2^k$, then $R[R[i]]$ is the 2^k-th ancestor of $R[i]$.

When we update $R[i]$ with $R[R[i]]$ (Step 4), from a. and b. we derive 1. and 2, respectively.

Parallel prefix sum on trees

PARALLEL PREFIX SUM ON TREES
Input: Two arrays P and W of n elements, where element i corresponds to node i in a forest, $P[i]$ is the parent of i and $W[i]$ is the weight of i (we assume that the weight of a root is 0).
Output: Update W so that $W[i]$ contains the sum of the weights of all the elements along the path from node i towards its root.

EXAMPLE:



index	1	2	3	4	5	6	7	8
P	5	3	3	8	3	7	4	8

index	1	2	3	4	5	6	7	8
W	3	4	0	2	-3	2	3	0

index	1	2	3	4	5	6	7	8
new W	0	4	0	2	-3	7	5	0

Parallel prefix sum on trees

PARALLEL PREFIX SUM ON TREES
Input: Two arrays P and W of n elements, where element i corresponds to node i in a forest, $P[i]$ is the parent of i and $W[i]$ is the weight of i (we assume that the weight of a root is 0).
Output: Update W so that $W[i]$ contains the sum of the weights of all the elements along the path from node i towards its root.

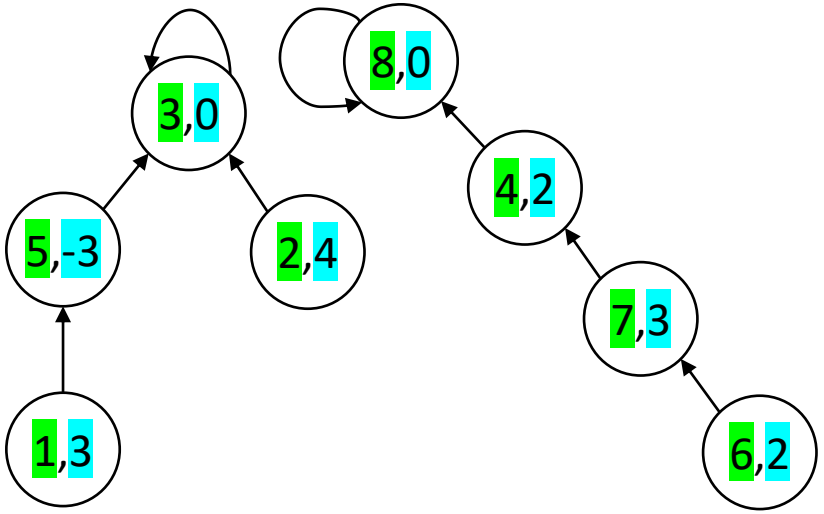
ALGORITHM Parallel_Prefix_Sum_On_Trees

```
for 1 ≤ i ≤ n pardo
    R[i] ← P[i]
    while(R[i] ≠ R[R[i]]) do
        W[i] ← W[i] + W[R[i]]
        R[i] ← R[R[i]]
```

An example

ALGORITHM Parallel_Prefix_Sum_On_Trees

1. **for** $1 \leq i \leq n$ **pardo**
2. $R[i] \leftarrow P[i]$
3. **while**($R[i] \neq R[R[i]]$) **do**
4. $W[i] \leftarrow W[i] + W[R[i]]$
5. $R[i] \leftarrow R[R[i]]$



index	1	2	3	4	5	6	7	8
P	5	3	3	8	3	7	4	8

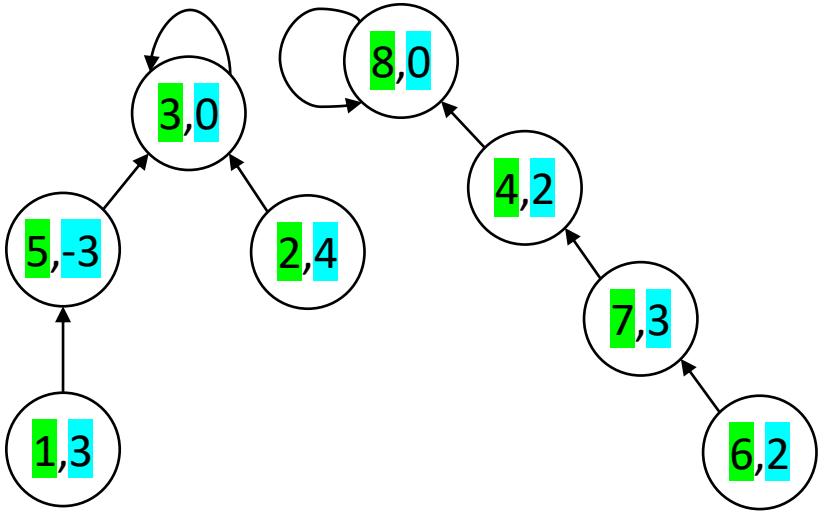
index	1	2	3	4	5	6	7	8
W	3	4	0	2	-3	2	3	0

index	1	2	3	4	5	6	7	8
R	5	3	3	8	3	7	4	8

An example

ALGORITHM Parallel_Prefix_Sum_On_Trees

1. **for** $1 \leq i \leq n$ **pardo**
2. $R[i] \leftarrow P[i]$
3. **while**$(R[i] \neq R[R[i]])$ **do**
4. $W[i] \leftarrow W[i] + W[R[i]]$
5. $R[i] \leftarrow R[R[i]]$



index	1	2	3	4	5	6	7	8
P	5	3	3	8	3	7	4	8

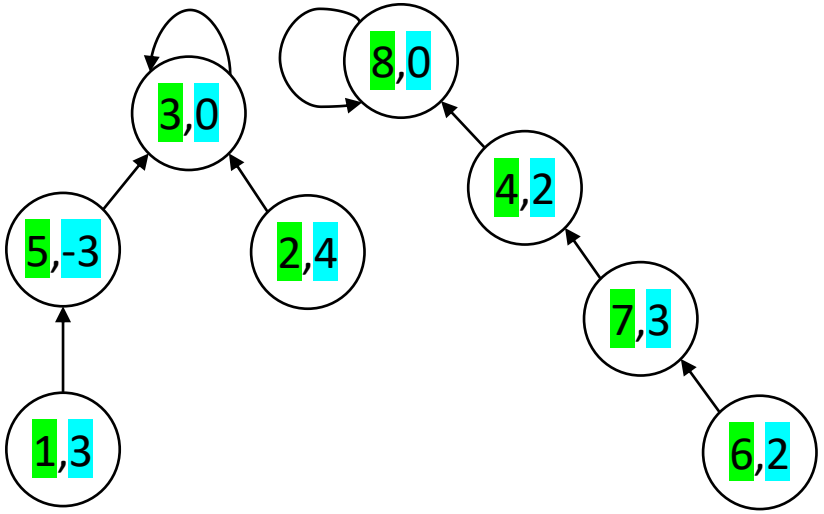
index	1	2	3	4	5	6	7	8
W	3	4	0	2	-3	2	3	0

index	1	2	3	4	5	6	7	8
R	5	3	3	8	3	7	4	8

An example

ALGORITHM Parallel_Prefix_Sum_On_Trees

1. **for** $1 \leq i \leq n$ **pardo**
2. $R[i] \leftarrow P[i]$
3. **while**$(R[i] \neq R[R[i]])$ **do**
4. $W[i] \leftarrow W[i] + W[R[i]]$
5. $R[i] \leftarrow R[R[i]]$



index	1	2	3	4	5	6	7	8
P	5	3	3	8	3	7	4	8

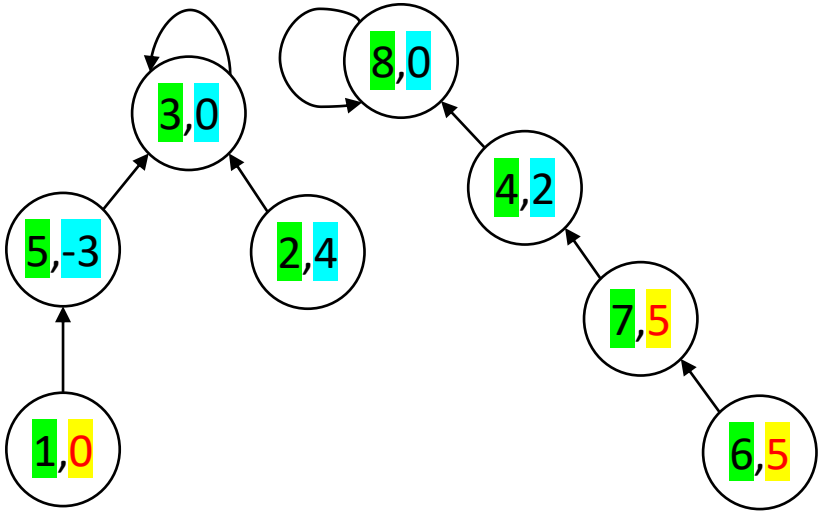
index	1	2	3	4	5	6	7	8
W	3	4	0	2	-3	2	3	0

index	1	2	3	4	5	6	7	8
R	5	3	3	8	3	7	4	8

An example

ALGORITHM Parallel_Prefix_Sum_On_Trees

1. **for** $1 \leq i \leq n$ **pardo**
2. $R[i] \leftarrow P[i]$
3. **while**$(R[i] \neq R[R[i]])$ **do**
4. $W[i] \leftarrow W[i] + W[R[i]]$
5. $R[i] \leftarrow R[R[i]]$



index	1	2	3	4	5	6	7	8
P	5	3	3	8	3	7	4	8

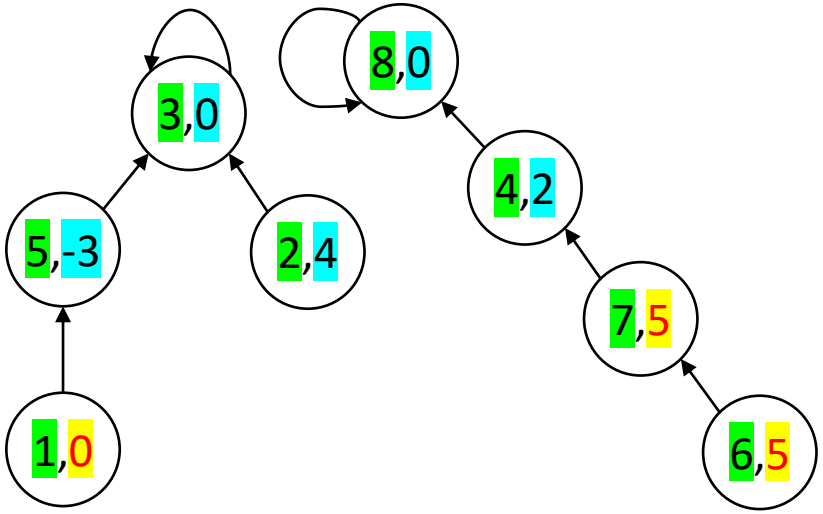
index	1	2	3	4	5	6	7	8
W	0	4	0	2	-3	5	5	0

index	1	2	3	4	5	6	7	8
R	5	3	3	8	3	7	4	8

An example

ALGORITHM Parallel_Prefix_Sum_On_Trees

1. **for** $1 \leq i \leq n$ **pardo**
2. $R[i] \leftarrow P[i]$
3. **while**$(R[i] \neq R[R[i]])$ **do**
4. $W[i] \leftarrow W[i] + W[R[i]]$
5. $R[i] \leftarrow R[R[i]]$



index	1	2	3	4	5	6	7	8
P	5	3	3	8	3	7	4	8

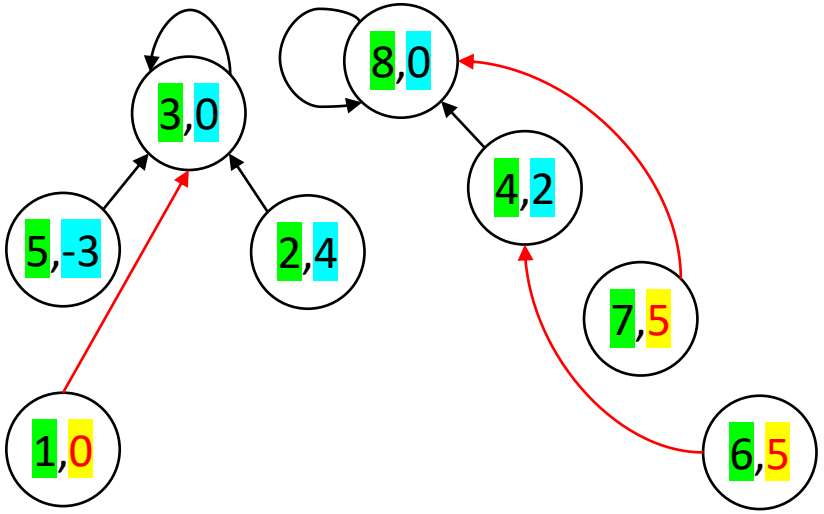
index	1	2	3	4	5	6	7	8
W	0	4	0	2	-3	5	5	0

index	1	2	3	4	5	6	7	8
R	5	3	3	8	3	7	4	8

An example

ALGORITHM Parallel_Prefix_Sum_On_Trees

1. **for** $1 \leq i \leq n$ **pardo**
2. $R[i] \leftarrow P[i]$
3. **while**$(R[i] \neq R[R[i]])$ **do**
4. $W[i] \leftarrow W[i] + W[R[i]]$
5. $R[i] \leftarrow R[R[i]]$



index	1	2	3	4	5	6	7	8
P	5	3	3	8	3	7	4	8

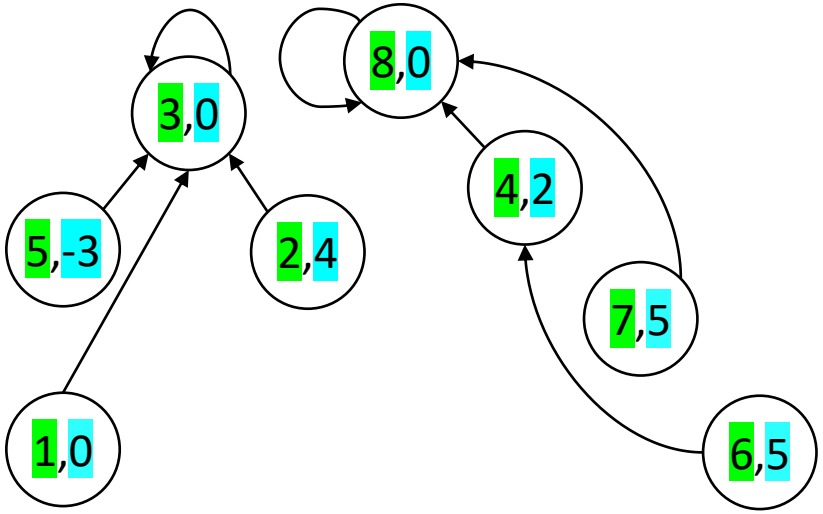
index	1	2	3	4	5	6	7	8
W	0	4	0	2	-3	5	5	0

index	1	2	3	4	5	6	7	8
R	3	3	3	8	3	4	8	8

An example

ALGORITHM Parallel_Prefix_Sum_On_Trees

1. **for** $1 \leq i \leq n$ **pardo**
2. $R[i] \leftarrow P[i]$
3. **while**$(R[i] \neq R[R[i]])$ **do**
4. $W[i] \leftarrow W[i] + W[R[i]]$
5. $R[i] \leftarrow R[R[i]]$



index	1	2	3	4	5	6	7	8
P	5	3	3	8	3	7	4	8

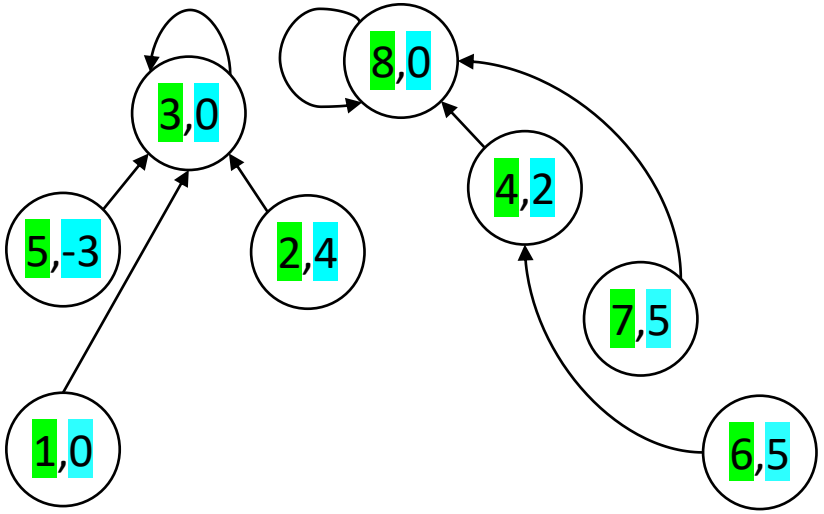
index	1	2	3	4	5	6	7	8
W	0	4	0	2	-3	5	5	0

index	1	2	3	4	5	6	7	8
R	3	3	3	8	3	4	8	8

An example

ALGORITHM Parallel_Prefix_Sum_On_Trees

1. **for** $1 \leq i \leq n$ **pardo**
2. $R[i] \leftarrow P[i]$
3. **while**$(R[i] \neq R[R[i]])$ **do**
4. $W[i] \leftarrow W[i] + W[R[i]]$
5. $R[i] \leftarrow R[R[i]]$



index	1	2	3	4	5	6	7	8
P	5	3	3	8	3	7	4	8

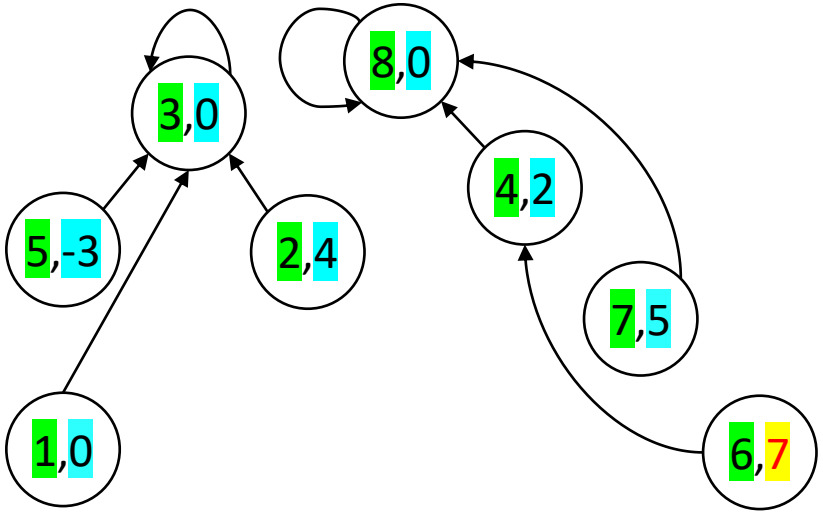
index	1	2	3	4	5	6	7	8
W	0	4	0	2	-3	5	5	0

index	1	2	3	4	5	6	7	8
R	3	3	3	8	3	4	8	8

An example

ALGORITHM Parallel_Prefix_Sum_On_Trees

1. **for** $1 \leq i \leq n$ **pardo**
2. $R[i] \leftarrow P[i]$
3. **while**$(R[i] \neq R[R[i]])$ **do**
4. $W[i] \leftarrow W[i] + W[R[i]]$
5. $R[i] \leftarrow R[R[i]]$



index	1	2	3	4	5	6	7	8
P	5	3	3	8	3	7	4	8

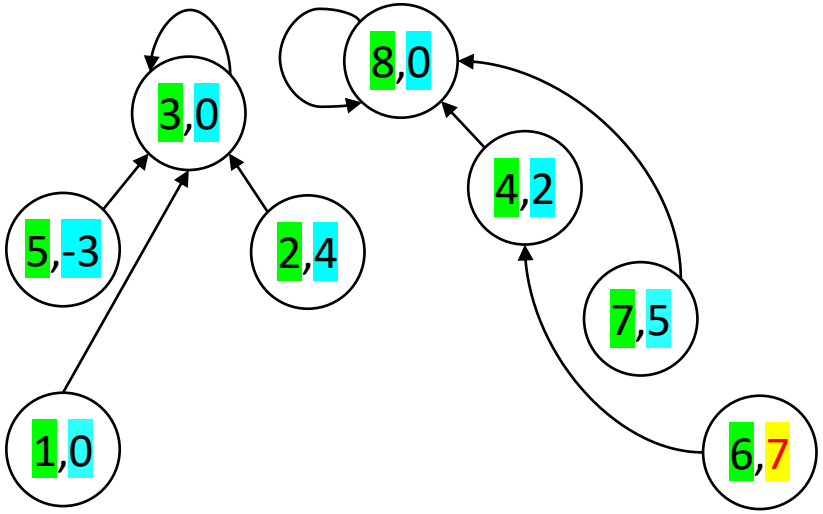
index	1	2	3	4	5	6	7	8
W	0	4	0	2	-3	7	5	0

index	1	2	3	4	5	6	7	8
R	3	3	3	8	3	4	8	8

An example

ALGORITHM Parallel_Prefix_Sum_On_Trees

1. **for** $1 \leq i \leq n$ **pardo**
2. $R[i] \leftarrow P[i]$
3. **while**$(R[i] \neq R[R[i]])$ **do**
4. $W[i] \leftarrow W[i] + W[R[i]]$
5. $R[i] \leftarrow R[R[i]]$



index	1	2	3	4	5	6	7	8
P	5	3	3	8	3	7	4	8

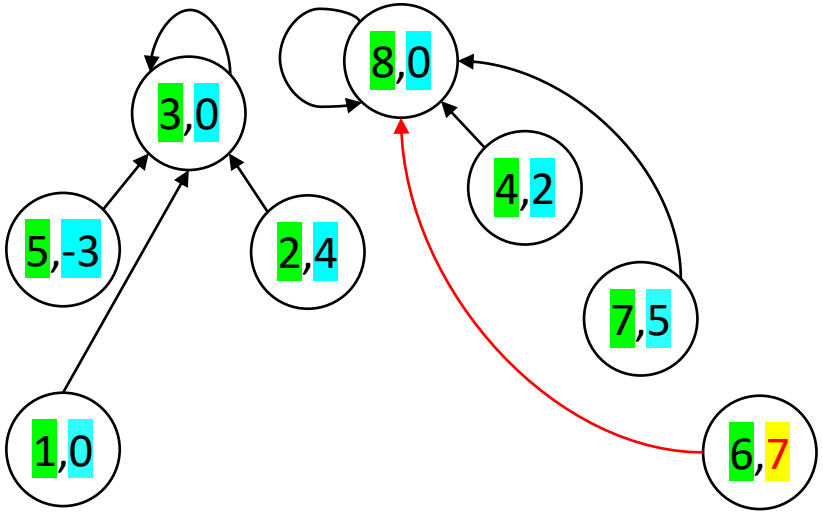
index	1	2	3	4	5	6	7	8
W	0	4	0	2	-3	7	5	0

index	1	2	3	4	5	6	7	8
R	3	3	3	8	3	4	8	8

An example

ALGORITHM Parallel_Prefix_Sum_On_Trees

1. **for** $1 \leq i \leq n$ **pardo**
2. $R[i] \leftarrow P[i]$
3. **while**$(R[i] \neq R[R[i]])$ **do**
4. $W[i] \leftarrow W[i] + W[R[i]]$
5. $R[i] \leftarrow R[R[i]]$



index	1	2	3	4	5	6	7	8
P	5	3	3	8	3	7	4	8

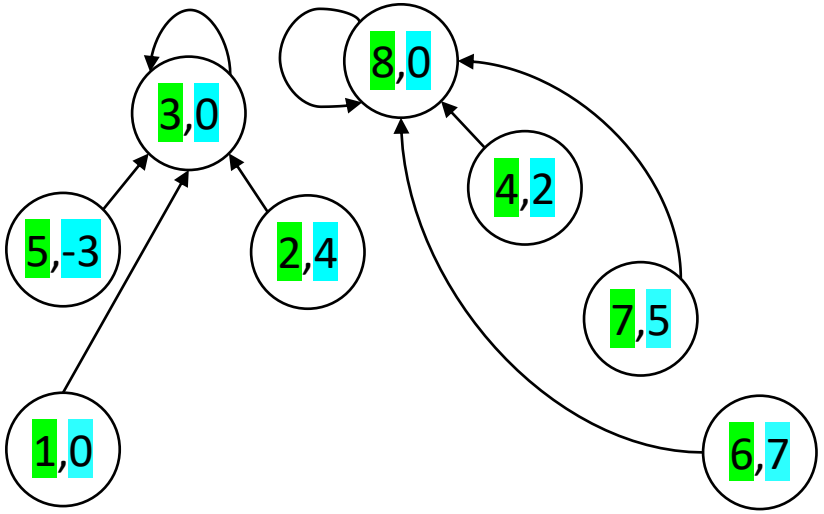
index	1	2	3	4	5	6	7	8
W	0	4	0	2	-3	7	5	0

index	1	2	3	4	5	6	7	8
R	3	3	3	8	3	8	8	8

An example

ALGORITHM Parallel_Prefix_Sum_On_Trees

1. **for** $1 \leq i \leq n$ **pardo**
2. $R[i] \leftarrow P[i]$
3. **while**$(R[i] \neq R[R[i]])$ **do**
4. $W[i] \leftarrow W[i] + W[R[i]]$
5. $R[i] \leftarrow R[R[i]]$



index	1	2	3	4	5	6	7	8
P	5	3	3	8	3	7	4	8

index	1	2	3	4	5	6	7	8
W	0	4	0	2	-3	7	5	0

index	1	2	3	4	5	6	7	8
R	3	3	3	8	3	8	8	8

The analysis

PARALLEL PREFIX SUM ON TREES
Input: Two arrays P and W of n elements, where element i corresponds to node i in a forest, $P[i]$ is the parent of i and $W[i]$ is the weight of i (we assume that the weight of a root is 0).
Output: Update W so that $W[i]$ contains the sum of the weights of all the elements along the path from node i towards its root.

ALGORITHM Parallel_Prefix

```
1.  for 1 ≤ i ≤ n pardo
2.      S[i] ← P[i]
3.      while(S[i] ≠ S[S[i]]) do
4.          W[i] ← W[i] + W[S[i]]
5.          S[i] ← S[S[i]]
```

- h: maximum among heights of all trees in the forest
- Time $O(\log h)$
- Work $O(n \log h)$ (not work optimal)
- CREW PRAM algorithm
  - Pointer jumping runs on a CREW PRAM
  - No concurrent writes on same variable

Check your understanding

THEOREM: Given a forest of rooted trees with n nodes, the algorithm Parallel_Prefix_Sum_On_Trees computes, for each node i, the sum of the weights of all the elements along the path from node i towards its root. The algorithm runs on a CREW PRAM in time $O(\log h)$ and work $O(n \log h)$, where h is the maximum among the heights of all trees in the forest.

PROOF: A good exercise.

OBSERVATION: The Parallel_Prefix_Sum_On_Trees algorithm is not work optimal as there is a sequential algorithm that runs in $O(n)$ time. Can you provide the pseudocode of this algorithm?